Project Report
on

QueryQuill – A Smart Study Buddy
at
U. V. Patel College of Engineering

Internal Guide :
Prof. Karan D. Modh

Prepared By:
Mr. Dhairya Thaker(23012021007)
Mr. Harshil Prajapati(23012011088)
Mr. Shlok Patel(23012011111)
Mr. Meet Suthar(23012011117)

B.Tech. Semester VI
(Computer Engineering)

December–April, 2025–2026

Submitted to,
Department of Computer Engineering
Faculty of Engineering & Technology
Ganpat University, Ganpat Vidyanagar, Mehsana - 384012

U.V.PATEL COLLEGE OF ENGINEERING

18/04/2026

CERTIFICATE

TO WHOM SO EVER IT MAY CONCERN

This is to certify that Mr. Dhairya Thaker (23012021007) student of **B.Tech. Semester VI (Information Technology)** has completed his full semester Capstone Project-II work titled "**QueryQuill – A Smart Study Buddy**" satisfactorily in partial fulfillment of the requirement of Bachelor of Technology degree of Computer Engineering of Ganpat University, Ganpat Vidyanagar, Mehsana in the year 2025-2026.

Prof. Karan D. Modh
College Project Guide

Dr. Devang Pandya
Head of the Department
Information Technology

U.V.PATEL COLLEGE OF ENGINEERING

18/04/2026

CERTIFICATE

TO WHOM SO EVER IT MAY CONCERN

This is to certify that Mr. Meet Suthar (23012011117) student of **B.Tech. Semester VI (Computer Engineering)** has completed his full semester Capstone Project-II work titled "**QueryQuill – A Smart Study Buddy**" satisfactorily in partial fulfillment of the requirement of Bachelor of Technology degree of Computer Engineering ofGanpat University, Ganpat Vidyanagar, Mehsana in the year 2025-2026.

Prof. Karan D. Modh
College Project Guide

Dr. Paresh M. Solanki
Head of the Department
Computer Engineering

U.V. PATEL COLLEGEOF ENGINEERING

18/04/2026

CERTIFICATE

TO WHOM SO EVER IT MAY CONCERN

This is to certify that Mr. Shlok Patel (23012011111) student of **B.Tech. Semester VI (Computer Engineering)** has completed his full semester Capstone Project-II work titled "**QueryQuill – A Smart Study Buddy**" satisfactorily in partial fulfillment of the requirement of Bachelor of Technology degree of Computer Engineering ofGanpat University, Ganpat Vidyanagar, Mehsana in the year 2025-2026.

Prof. Karan D. Modh
College Project Guide

Dr. Paresh M. Solanki
Head of the Department
Computer Engineering

U.V. PATEL COLLEGEOF ENGINEERING

18/04/2026

CERTIFICATE

TO WHOM SO EVER IT MAY CONCERN

This is to certify that Mr. Harshil Prajapati (23012011088) student of **B.Tech. Semester VI (Computer Engineering)** has completed his full semester Capstone Project-II work titled "**QueryQuill – A Smart Study Buddy**" satisfactorily in partial fulfillment of the requirement of Bachelor of Technology degree of Computer Engineering ofGanpat University, Ganpat Vidyanagar, Mehsana in the year 2025-2026.

Prof. Karan D. Modh
College Project Guide

Dr. Paresh M. Solanki
Head of the Department
Computer Engineering

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to all those who contributed to the successful completion of the Software Requirements Specification (SRS) for the project *QueryQuill – A Smart Study Buddy*.

We extend our heartfelt thanks to our project guide, **Prof. Karan D. Modh**, for her continuous guidance, constructive feedback, and valuable insights throughout the development of this document. Her support played a crucial role in shaping our understanding of system requirements and ensuring clarity and structure in this SRS.

We also express our gratitude to **Dr. Devang Pandya**, Head of the Information Technology Department, and **Dr. Paresh Solanki**, Head of the Computer Engineering Department, for providing the necessary academic support and an environment that encourages innovation and technical growth.

We would like to acknowledge all faculty members of the IT and CE departments for their guidance and the strong academic foundation they have provided, which has been essential in understanding the concepts related to software engineering, artificial intelligence, and natural language processing used in this project.

With sincere regards,
Dhairya Thaker
Harshil Prajapati
Shlok Patel
Meet Suthar

ABSTRACT

QueryQuill is an AI-powered educational platform designed to convert static learning resources into interactive and structured study materials. Traditional learning methods often rely on passive consumption of textbooks, documents, and video lectures, requiring significant manual effort to extract meaningful insights. QueryQuill addresses this limitation by enabling users to upload text-based content, including documents, notes, PDFs, and YouTube video links, and automatically transforming them into engaging learning tools.

The system leverages advanced Natural Language Processing (NLP) techniques using multiple specialized Transformer-based models. It employs T5 for context-aware question generation, DistilBART for text summarization, and FLAN-T5 to power a conversational Socratic tutoring system. By deeply analyzing content, the platform generates relevant and structured educational outputs that enhance comprehension and retention.

Key features of QueryQuill include automated assessment generation (both multiple-choice and descriptive questions), intelligent summarization with slide creation, an AI-driven Socratic tutor that promotes critical thinking through guided hints, and interactive knowledge maps that visually represent relationships between topics and subtopics. These features collectively support active learning and help users identify knowledge gaps effectively.

The platform is built using a modern and scalable technology stack, with a React/Vite frontend, FastAPI backend, and MongoDB database. Its flexible design allows customization for different use cases, including academic learning, self-study, competitive exam preparation, and corporate training.

In essence, QueryQuill provides an end-to-end solution for transforming passive educational content into an interactive learning experience, focusing on improving conceptual understanding, analytical skills, and knowledge retention rather than rote memorization.

INDEX

CHAPTER 1: INTRODUCTION 1
1.1 Problem Statement 1
1.2 Objective 1
1.3 Purpose 2
1.4 Project Scope 3
1.5 Overview of the Document 4
CHAPTER 2: PROJECT SCOPE 5
2.1 Overview 5
2.2 Literature Review 5
2.3 Case Studies 6
2.4 Domain Analysis 6
2.5 Problem Validation 7
2.6 Review of Existing Systems 8
2.7 Feasibility Study 8
2.8 Objectives of the Project 9
2.9 Scope of the System 9
CHAPTER:3 FEASIBILITY ANALYSIS 10
3.1 Technical Feasibility 10
3.2 Time Schedule Feasibility 11
3.3 Operational Feasibility 12
3.4 Implementation Feasibility 13
3.5 Economic Feasibility 13
CHAPTER 4: SOFTWARE & HARWARE REQUIREMENTS 15
4.1 Software Requirements 15
4.1.1 System Software Requirements 15
4.1.2 Frontend Requirements 15
4.1.3 Backend Requirements 15
4.1.4 NLP and Processing Libraries 16
4.1.5 Database Requirements 16
4.1.6 Development Tools 16
4.2 Hardware Requirements 16
4.2.1 Client-Side Requirements 16

4.2.2 Server-Side Requirements 17
4.2.3 Deployment Environment 17
4.3 Performance Considerations 17
4.4 Scalability Considerations 17
CHAPTER 5: PROJECT MODEL 18
5.1 Agile Methodology in QueryQuill 18
5.2 Phases of Agile Model 18
5.2.1 Requirement Gathering 18
5.2.2 System Design 19
5.2.3 Implementation (Sprint Development) 19
5.2.4 Continuous Testing 19
5.2.5 Deployment 20
5.2.6 Maintenance and Iteration 20
5.3 Advantages of Agile for QueryQuill 20
5.4 Process Flow Summary 21
CHAPTER 6: PROJECT PLAN 22
6.1 Project Milestones 22
6.2 Project Timeline 22
6.3 Resource Allocation 23
6.3.1 Human Resources 23
6.3.2 Technical Resources 23
6.3.3 Financial Resources 23
6.4 Risk Management 23
6.4.1 Technical Risks 23
6.4.2 Security Risks 23
6.5 Quality Assurance (QA) 23
6.5.1 Testing Strategy 23
6.5.2 Performance Metrics 24
6.6 Maintenance and Support 24
6.6.1 System Maintenance 24
6.6.2 User Support 24
CHAPTER 7: SYSTEM DESIGN 25
7.1 Design Strategy 25
7.2 System Diagrams (UML) 25

7.2.1 USE CASE DIAGRAM 26
7.2.2 CLASS DIAGRAM 27
7.2.3 SEQUENCE DIAGRAM 28
7.2.4 ACTIVITY DIAGRAM 29
7.2.5 E-R DIAGRAM 30
7.4 List of Tables & Data Dictionary 31
7.4.1 Collection: users 31
7.4.2 Collection: questions 31
7.4.3 Collection: exams 31
7.5 Data Flow Diagram (DFD) 32
7.5.1 Level 0 DFD (Context Level) 32
7.5.2 Level 1 DFD 32
7.5.3 Level 2 DFD 34
7.6 User Interface (UI) Design Outline 34
CHAPTER 8: IMPLEMENTATION DETAILS 36
8.1 Algorithm of Implementation 36
8.2 Flowchart of Implementation 37
8.3 Actual Program Code 38
8.3.1 NLTK Heuristic Answer Identification 38
8.3.2 Hugging Face T5 NLP Inference Loop 38
8.3.3 Socratic Tutor Endpoint (FLAN-T5) 39
CHAPTER 9: TESTING 40
9.1 Testing Approach 40
9.2 Basic Test Cases 40
9.3 Result Summary 40
CHAPTER 10: USER MANUAL 41
10.1 System Installation & Setup 41
10.2 System Usage 41
10.3 Application Screenshots 42
10.3.1 Login / Sign Up Page 42
10.3.2 Student Dashboard 43
10.3.3 Teacher Dashboard 43
10.3.4 AI Dictionary 44
10.3.5 AI Summarizer 44

10.3.6 Exam Creator 45
10.3.7 Exam Attempt Portal 45
CHAPTER 11: CONCLUSION AND FUTURE WORK 46
11.1 Conclusion 46
11.2 Future Enhancements 46
CHAPTER 12: ANNEXURE 47
12.1 Glossary of Terms and Abbreviations 47
12.2 References 47
12.3 Tools and Technologies 48
12.4 About the College 48

LIST OF FIGURES

Figure 7.2.1 Use Case Diagram 26
Figure 7.2.2 Class Diagram 27
Figure 7.2.3 Sequence Diagram 28
Figure 7.2.4 Activity Diagram 29
Figure 7.2.5 E-R Diagram 30
Figure 7.5.1 Level 0 DFD6 32
Figure 7.5.2 Level 1 DFD 33
Figure 7.5.3 Level 2 DFD8 34
Figure 8.2 Flowchart of Implementation 37
Figure 10.3.1 Login/SignUp Page 42
Figure 10.3.2 Student Dashboard 43
Figure 10.3.3 Teacher Dashboard 43
Figure 13.4 AI Dictionary 44
Figure 10.3.5 AI Summarizer 44
Figure 10.3.7 Exam Attempt Portal 45

LIST OF TABLES

Table 4.2.1 Client-Side Requirements 16
Table 4.2.2 Server-Side Requirements 17
Table 6.1 Project Milestones 22
Table 6.2 Project Timeline 22
Table 6.3.1 Human Resources 23
Table 6.4.1 Technical Risks 23
Table 6.5.2 Performance Metrics 24
Table 7.4.1 Collection: users 31
Table 7.4.2 Collection: questions 31
Table 7.4.3 Collection: exams 32

VISION AND MISSION OF GANPAT UNIVERSITY

Vision:
It shall be the constant endeavor of Ganpat University to meet the educational needs of the youth in the areas of professional studies and provide state-of-the-art learning opportunities along with inculcation of values of commitment and uprightness.

Mission:
Seek, search and offer programs that lead to symbiotic emergence of academic excellence and industrial relevance in education and research.

Quality Statement:
It shall be the constant endeavour of the University to deliver global standards of excellence in all its core and allied operations and remaining accountable for national values and objectives of establishment through process of self evaluation and continuous improvement.

CHAPTER 1: INTRODUCTION

1.1 Problem Statement

In the current digital learning environment, the availability of educational content has increased exponentially. Students and educators have access to textbooks, research papers, lecture notes, and video-based learning platforms. However, most of these resources remain **static, lengthy, and unstructured**, making it difficult for learners to extract meaningful insights efficiently.

Students often face challenges such as:

- Difficulty in identifying key concepts from large volumes of content
- Lack of structured practice material for self-assessment
- Passive learning without interaction or engagement
- Inability to track understanding and knowledge gaps

At the same time, educators and trainers encounter significant challenges in:

- Manually creating question banks and assessments
- Preparing concise summaries and presentation slides
- Converting raw content into structured teaching material
- Scaling content creation for large groups of learners

This manual process is **time-consuming, repetitive, and inefficient**, especially in environments that demand personalization and rapid content delivery.
To address these limitations, **QueryQuill** proposes an AI-driven solution that transforms passive educational content into **interactive, structured, and personalized learning resources**. By leveraging advanced Natural Language Processing (NLP) techniques, the system automates the extraction, analysis, and transformation of content into meaningful educational outputs.

1.2 Objective

The primary objective of QueryQuill is to design and develop an **intelligent, end-to-end educational platform** capable of converting raw learning materials into interactive study tools using AI and NLP techniques.

The system aims to achieve the following objectives:

- **Automated Question Generation**
  Generate context-aware multiple-choice questions (MCQs) and descriptive questions using advanced transformer-based models such as T5.

- **Content Summarization and Slide Creation**
  Convert lengthy documents and transcripts into concise summaries and structured presentation slides using models like DistilBART.
- **Socratic Learning Support**
  Implement an AI-powered tutor that provides hints and guided assistance

instead of direct answers, encouraging analytical thinking and problem-solving.

- **Knowledge Visualization**
  Create dynamic knowledge maps that represent relationships between topics and subtopics, enabling users to identify learning gaps.

- **Multi-Format Content Handling**
  Support various input formats including PDFs, text documents, and YouTube video links for flexible content ingestion.

- **User Customization and Accessibility**
  Allow users to customize difficulty levels, question types, and output formats to suit individual learning needs.

- **Productivity Enhancement**
  Reduce the workload of educators by automating the generation of assessments, summaries, and teaching materials.

1.3 Purpose

The purpose of QueryQuill is to bridge the gap between **traditional content consumption** and **modern interactive learning methodologies**. Instead of requiring users to passively read or watch content, the system transforms learning into an **active and engaging process**.
QueryQuill is designed to serve multiple user groups:

- **Students and Self-Learners**
  Enables learners to test their understanding through generated questions, summaries, and guided assistance, improving retention and comprehension.

- **Teachers and Educators**
  Assists in quickly generating quizzes, summaries, and presentation materials, reducing manual effort and improving teaching efficiency.

- **Coaching Institutes and E-Learning Platforms**
  Supports scalable creation of standardized and high-quality learning resources from existing content.

- **Working Professionals and Corporate Trainers**
  Helps convert technical documents, training materials, and recorded sessions into structured learning formats.

By automating the transformation of content into interactive resources, QueryQuill enhances:

- Conceptual clarity
- Learning efficiency
- Engagement and retention
- Teaching productivity

1.4 Project Scope

QueryQuill is developed as a **comprehensive content transformation pipeline** that integrates document processing, NLP-based analysis, and intelligent output generation.
The scope of the system includes:

- **Content Ingestion and Processing**
  The system accepts input in multiple formats such as PDF files, raw text, and YouTube URLs. It extracts and preprocesses textual content for further analysis.

- **Multi-Model NLP Framework**
  QueryQuill utilizes multiple transformer-based models:
  - T5 for question generation
  - DistilBART for summarization
  - FLAN-T5 for conversational tutoring

  This multi-model approach ensures high-quality, context-aware outputs.

- **Automated Learning Resource Generation**
  The platform generates:
  - Multiple-choice questions (MCQs)
  - Descriptive questions
  - Summaries
  - Presentation slides

- **Interactive Socratic Tutor**
  Provides intelligent hints and feedback based on user responses, promoting deeper understanding rather than rote memorization.

- **Knowledge Mapping System**
  Visualizes relationships between topics through interconnected graphs, helping users understand content structure.

- **SystemArchitecture**
  The platform is built using:
  - React/Vite for frontend
  - FastAPI for backend
  - MongoDB for data storage

  This ensures scalability, responsiveness, and efficient processing.

The current scope excludes:
- Advanced predictive analytics for long-term learning behavior
- Fully offline functionality (requires internet/API access)
- Real-time collaborative editing features

1.5 Overview of the Document

This chapter provides a comprehensive introduction to the QueryQuill system, including its problem context, objectives, purpose, and scope.

The remainder of this Software Requirements Specification (SRS) document is structured as follows:

- **Chapter 2:** System Requirements and Specifications
- **Chapter 3:** Functional and Non-Functional Requirements
- **Chapter 4:** Development Methodology (Agile Model)
- **Chapter 5:** System Architecture and AI Model Design
- **Chapter 6:** Database Design and Use Case Diagrams
- **Chapter 7:** Implementation Details and Modules
- **Chapter 8:** Testing and Validation Strategies
- **Chapter 9:** Conclusion and Future Enhancements

QueryQuill aims to provide a **scalable, intelligent, and user-centric learning ecosystem** that enhances both teaching and learning experiences through automation and AI-driven insights.

CHAPTER 2: PROJECT SCOPE

2.1 Overview

QueryQuill is an AI-powered educational platform designed to convert passive learning resources into structured, interactive, and personalized study materials. Traditional educational content—such as textbooks, PDFs, research papers, and video lectures—often lacks interactivity, making it difficult for learners to actively engage with the material and assess their understanding.

QueryQuill addresses this limitation by integrating advanced Natural Language Processing (NLP) techniques to automatically process and transform raw content into meaningful educational outputs. The system leverages multiple specialized transformer-based models, including:

- **T5** for context-aware question generation
- **DistilBART** for text summarization
- **FLAN-T5** for conversational and Socratic tutoring

By combining these models into a unified pipeline, QueryQuill enables deep semantic understanding of content and produces high-quality learning artifacts such as assessments, summaries, and knowledge maps.

The platform enhances:

- Learning efficiency through automation
- Engagement through interactivity
- Productivity by reducing manual effort

2.2 Literature Review

The current ecosystem of AI-assisted document tools includes platforms such as ChatPDF, AskYourPDF, and PDF.ai. These systems primarily focus on:

- Extractive summarization
- Direct question-answering
- Conversational interaction with documents

While effective for quick information retrieval, these tools have several limitations:

- Lack of structured educational outputs (e.g., quizzes, slides)
- Minimal support for pedagogical design
- Limited customization of learning parameters
- Absence of guided learning approaches such as Socratic questioning
- Weak support for multimodal inputs (especially video transcripts)

Additionally, most systems rely on general-purpose models without domain-specific optimization for education, leading to shallow or generic outputs.
QueryQuill differentiates itself by:

- Using a **multi-model architecture** instead of a single generic model
- Focusing on **education-specific outputs** rather than generic responses

- Supporting **end-to-end workflow automation** (content → learning tools)
- Incorporating **Socratic learning principles**
- Providing **visual knowledge mapping**

This positions QueryQuill as a specialized EdTech solution rather than a general AI assistant.

2.3 Case Studies

1. **Massive Open Online Course Platforms (Coursera, edX)**

These platforms provide structured courses with predefined assessments. However:

- Content is **static and pre-authored**
- No support for **user-generated content transformation**
- Limited personalization

2. **General AI Assistants (ChatGPT, Claude)**

These tools are highly capable but:

- Require **manual prompting and user effort**
- Lack **consistent structured output generation**
- Do not provide integrated workflows for education

3. **AI-Based Quiz Generators**

Existing tools often:

- Use **keyword-based extraction**
- Generate **low-quality distractors**
- Focus on memorization rather than understanding

4. **Corporate Learning Systems**

Enterprise systems provide:

- Document storage and retrieval
- Basic training modules

But lack:

- Automated content transformation
- Intelligent assessment generation
- Interactive tutoring mechanisms

2.4 Domain Analysis

QueryQuill operates at the intersection of **Educational Technology (EdTech)** and **Applied Natural Language Processing (NLP)**.

Input Layer

- PDF documents
- Text files (TXT, DOCX)
- YouTube video transcripts

Processing Pipeline

1. Content extraction and preprocessing
2. Semantic chunking and structuring
3. Model-specific processing:
    - Question Generation
    - Summarization
    - Tutoring response generation
4. Aggregation and formatting of outputs

Output Layer

- Multiple-choice questions (MCQs)
- Descriptive questions
- Summaries
- Presentation slides
- Knowledge maps
- Socratic hints

Target Users

- Students and self-learners
- Educators and academic institutions
- Coaching centers
- Corporate training environments

Technology Stack

- Frontend: React + Vite
- Backend: FastAPI
- Database: MongoDB
- NLP Models: T5, DistilBART, FLAN-T5

2.5 Problem Validation

Analysis of real-world educational workflows highlights several critical issues:

Student Challenges

- Difficulty revising large volumes of content
- Lack of structured practice and feedback
- Passive consumption without engagement
- Over-reliance on direct answers

Educator Challenges

- Time-intensive manual creation of quizzes and slides
- Difficulty scaling content across multiple learners
- Fragmented use of tools for different tasks

System-Level Gaps

- Lack of integrated platforms combining multiple functionalities
- Absence of adaptive and guided learning systems
- Limited automation in content transformation

These challenges validate the need for a unified, intelligent platform like QueryQuill.

2.6 Review of Existing Systems

Existing solutions can be broadly categorized as:

1. **Basic Quiz Generators**

- Template-based or keyword-driven
- Low contextual understanding
- Limited adaptability

2. **General AI Tools**

- Strong in explanation and summarization
- Weak in structured educational workflows
- Lack automation and scalability

QueryQuill bridges this gap by providing a **complete, automated educational pipeline**.

2.7 Feasibility Study

1. **Technical Feasibility**

- Uses proven and scalable technologies (React, FastAPI, MongoDB)
- Leverages open-source NLP models from Hugging Face
- Supports asynchronous processing for performance efficiency
- Offloads heavy computation to backend systems

2. **Economic Feasibility**

- Minimal licensing cost due to open-source tools
- Flexible deployment (cloud or local servers)
- High return in terms of productivity and efficiency

3. **Operational Feasibility**

- Simple and intuitive user interface
- Minimal training required
- Suitable for both technical and non-technical users

4. **Environmental Feasibility**

- Reduces paper usage through digital assessments
- Promotes efficient resource utilization via cloud computing

2.8 Objectives of the Project

The key objectives of QueryQuill are:

- Automate the transformation of learning materials into structured outputs
- Support multiple input formats including documents and video transcripts
- Generate high-quality assessments and summaries
- Promote active learning through Socratic guidance
- Visualize knowledge structures using interactive maps
- Reduce manual workload for educators

2.9 Scope of the System

System Will:

- Accept inputs in PDF, DOCX, TXT, and YouTube URL formats
- Extract and preprocess textual content
- Generate context-aware MCQs and descriptive questions
- Produce summaries and presentation slides
- Provide an AI-based Socratic tutoring interface
- Visualize topic relationships through knowledge maps
- Allow user customization of output parameters

System Will Not:

- Support real-time collaborative editing
- Act as a cloud storage platform for large media files
- Fully automate grading of handwritten responses
- Replace human educators; it acts as a support tool

CHAPTER:3 FEASIBILITY ANALYSIS

A feasibility analysis is a critical phase in the Software Development Life Cycle (SDLC) that evaluates whether a proposed system is practical, sustainable, and worth implementing. It ensures that the system can be developed within existing constraints related to technology, time, cost, and operational usability.

For QueryQuill, this analysis validates that an AI-powered educational platform—built on Natural Language Processing (NLP) and modern web technologies—can be successfully designed, deployed, and maintained with available resources.

3.1 Technical Feasibility

Technical feasibility assesses whether the required technology stack, tools, and infrastructure are sufficient to implement the system efficiently and reliably.

QueryQuill is **highly technically feasible** due to its use of proven, industry-standard technologies:

Frontend Layer

The system uses **React with Vite**, enabling:

- Fast rendering and efficient component-based architecture
- Responsive and interactive UI (knowledge maps, slides, dashboards)
- Seamless user experience across devices

Backend Layer

The backend is developed using **Python with FastAPI**, which provides:

- High-performance asynchronous request handling
- Efficient API communication between frontend and NLP models
- Scalability for concurrent users

This is particularly critical because NLP model inference is computationally intensive and must not block server operations.

Natural Language Processing (NLP)

QueryQuill integrates multiple open-source transformer models:

- **T5 (t5-base-qg-hl)** → Question Generation
- **DistilBART** → Text Summarization
- **FLAN-T5** → Socratic Tutoring

These models are:

- Pre-trained and widely validated
- Optimized for performance and accuracy
- Suitable for domain-specific adaptation

Database Layer

The system uses **MongoDB (NoSQL)**:

- Flexible schema for storing structured/unstructured data
- Efficient handling of JSON-based outputs (questions, summaries, maps)
- Easy scalability without rigid migrations

Content Ingestion

- PDF parsing via PyPDF
- Video transcript extraction via YouTube APIs

Technical Conclusion

The combination of:

- Mature frameworks
- Open-source NLP models
- Scalable architecture
-

ensures that the system is **fully implementable with current technology** and does not depend on experimental or unstable components.

3.2 Time Schedule Feasibility

Time feasibility evaluates whether the project can be completed within the given academic or development timeline.

QueryQuill is **time-efficient and achievable** due to:

Use of Pre-Trained Models

- Eliminates the need for training models from scratch
- Saves significant time in data collection and training
- Focus shifts to integration and optimization

Modular Architecture

- Frontend and backend are decoupled
- Enables parallel development
- Reduces dependency bottlenecks

Agile Development Approach

The project can be divided into manageable iterations:

- **Sprint 1:** Content upload and parsing
- **Sprint 2:** Question generation module
- **Sprint 3:** Summarization and slide generation
- **Sprint 4:** Socratic tutor integration
- **Sprint 5:** Knowledge map and UI refinement

Time Conclusion

Given proper planning and task distribution, the system can be completed within standard academic timelines.

3.3 Operational Feasibility

Operational feasibility evaluates how effectively the system solves the identified problems and how easily it can be adopted by users.

QueryQuill demonstrates **strong operational feasibility**:

Ease of Use

- Simple workflow: upload → process → generate
- No technical expertise required
- Minimal learning curve

User-Centric Design

- Designed for both students and educators
- Supports real-world academic workflows
- Reduces cognitive and operational burden

Automation of Tasks

- Eliminates manual creation of quizzes and summaries
- Reduces repetitive work for educators
- Enables faster content preparation

Active Learning Support

- Socratic tutor encourages critical thinking
- Avoids passive answer consumption
- Aligns with modern pedagogy

Platform Independence

- Web-based application
- Works across desktops, laptops, and tablets
- No installation required

Operational Conclusion

The system integrates seamlessly into existing educational workflows and provides immediate, practical value.

3.4 Implementation Feasibility

Implementation feasibility evaluates whether the system can be realistically built, tested, and deployed.

QueryQuill is **highly implementable** due to:

Optimized Model Selection

- Uses lightweight models (distilled/small variants)
- Reduces computational overhead
- Enables deployment on affordable infrastructure

Hardware Requirements

- Does not require high-end GPU clusters
- Can run on:
    - CPU-based servers
    - Entry-level cloud GPU instances

Open-Source Ecosystem

- Libraries such as FastAPI, PyMongo, and NLP toolkits are well-documented
- Strong community support reduces development risk

Modular Design

- Independent modules for:
    - NLP processing
    - API handling
    - UI rendering
- Allows:
    - Easy debugging
    - Future upgrades (model replacement without system rewrite)

Implementation Conclusion

The system is practical to build and maintain, with minimal technical risk.

3.5 Economic Feasibility

Economic feasibility analyzes whether the benefits of the system outweigh its costs. QueryQuill demonstrates **high economic viability**:

Low Development Cost

- Uses free and open-source technologies:
    - React, Vite
    - FastAPI
    - MongoDB
    - Hugging Face models

No Recurring API Costs

- Avoids paid APIs (e.g., GPT, Claude)
- Eliminates usage-based billing risks

Affordable Deployment

- Can be hosted on:
    - Low-cost cloud servers
    - Institutional infrastructure

High Value Output

- Saves time for educators
- Provides scalable learning tools
- Reduces need for external tutoring resources

Cost-Benefit Advantage

- Initial setup cost is minimal
- Long-term benefits significantly outweigh costs

Economic Conclusion

The system delivers high utility at low cost, making it economically sustainable.

CHAPTER 4: SOFTWARE & HARWARE REQUIREMENTS

This chapter defines the software and hardware requirements necessary for the development, deployment, and operation of the QueryQuill system. The requirements are categorized into system software, application software, development tools, and hardware specifications for both client-side and server-side environments.

4.1 Software Requirements

The QueryQuill system is implemented as a web-based application following a client-server architecture. The software requirements are divided into the following categories:

4.1.1 System Software Requirements

These are the foundational software components required to run the system.

Component	Requirement
Operating System (Client)	Windows 10/11, macOS, Linux, Android, iOS
Operating System (Server)	Linux (Ubuntu 20.04/22.04 recommended)
Web Browser	Google Chrome, Mozilla Firefox, Microsoft Edge, Safari (latest versions)
Runtime Environment	Node.js (for frontend), Python 3.9+ (for backend)

Table 4.1.1 System Software Requirements

4.1.2 Frontend Requirements

The frontend is responsible for user interaction and presentation.

- HTML5, CSS3, JavaScript (ES6+)
  Used for structuring, styling, and implementing interactive UI behavior.
- React.js
  Component-based library for building dynamic and responsive user interfaces.
- Vite
  Modern build tool for faster development and optimized production builds.
- Tailwind CSS (Optional)
  Utility-first CSS framework for rapid UI styling and responsiveness.
- Axios / Fetch API
  Handles asynchronous communication with backend services.

4.1.3 Backend Requirements

The backend handles business logic, API communication, and AI processing.

- Python 3.9+
  Core programming language for backend and NLP integration.
- FastAPI
  High-performance web framework supporting asynchronous API handling.
- Uvicorn
  ASGI server used to deploy FastAPI applications.

4.1.4 NLP and Processing Libraries

These libraries enable intelligent data processing and AI functionality.

- Hugging Face Transformers
  Used for integrating models such as:
  - T5 → Question Generation
  - DistilBART → Summarization
  - FLAN-T5 → Socratic Tutor
- NLTK / Text Processing Libraries
  Used for tokenization, parsing, and text preprocessing.
- PyPDF / pdfplumber
  For extracting text from PDF documents.
- youtube_transcript_api
  For retrieving transcripts from YouTube videos.

4.1.5 Database Requirements

- MongoDB (NoSQL Database)
  - Stores user data, generated outputs, and session information
  - Supports flexible schema for JSON-like AI outputs
- PyMongo / Motor
  - Enables communication between backend and database
  - Supports asynchronous operations

4.1.6 Development Tools

These tools support development, testing, and version control.

- Visual Studio Code (VS Code) – IDE for development
- Git & GitHub – Version control and collaboration
- npm (Node Package Manager) – Frontend dependency management
- pip / virtualenv – Backend dependency management

4.2 Hardware Requirements

QueryQuill follows a thin-client architecture, where all heavy computations are performed on the server. This ensures accessibility across a wide range of user devices.

4.2.1 Client-Side Requirements

Minimum hardware requirements for end users:

Component	Minimum Requirement	Recommended
Processor	Dual-core 1.6 GHz	Quad-core 2.0 GHz+
RAM	2 GB	4 GB or higher
Storage	200 MB free space	500 MB+
Device Type	Laptop, Desktop, Tablet, Mobile	—
Browser	Latest modern browser	Chrome/Edge preferred

Table 4.2.1 Client-Side Requirements

KeyPoint:
No specialized hardware is required on the client side since all AI processing is handled remotely.

4.2.2 Server-Side Requirements

Server infrastructure must support AI model execution and concurrent users.

Component	Minimum Requirement	Recommended
Processor	Multi-core CPU (2.0 GHz+)	High-performance CPU
RAM	8 GB	16 GB or higher
Storage	50 GB SSD	100 GB+ SSD
GPU (Optional)	Not mandatory	NVIDIA T4 or equivalent
Network	Stable internet	High bandwidth (for scaling)
OS	Linux (Ubuntu)	Optimized ML environment

Table 4.2.2 Server-Side Requirements

4.2.3 Deployment Environment

The system can be deployed using:

- Cloud Platforms:
  - AWS
  - Google Cloud Platform
  - DigitalOcean
- Frontend Hosting:
  - Vercel
  - Netlify
- Backend Hosting:
  - Cloud VM instances or containerized environments

4.3 Performance Considerations

- Backend must handle asynchronous processing to avoid blocking during NLP tasks
- Model inference latency should be optimized using:
  - Model selection (distilled models)
  - Caching strategies
- System should support multiple concurrent users

4.4 Scalability Considerations

- Modular architecture allows:
  - Easy integration of new AI models
  - Horizontal scaling of backend services
- Database supports high-volume data storage and retrieval
- Cloud deployment enables dynamic scaling

CHAPTER 5: PROJECT MODEL

QueryQuill is developed using the **Agile Software Development Model**, an iterative and incremental approach that emphasizes flexibility, continuous feedback, and rapid delivery of functional components.

Unlike traditional linear models, Agile allows the system to evolve progressively through multiple development cycles (sprints). This is particularly important for QueryQuill due to the **non-deterministic and experimental nature of AI and NLP systems**, where outputs must be continuously evaluated and refined.

The Agile model ensures:

- Continuous improvement of NLP model outputs
- Rapid integration of new features
- Early validation of system functionality
- Adaptability to changing requirements

5.1 Agile Methodology in QueryQuill

The development of QueryQuill follows an iterative cycle consisting of multiple short sprints. Each sprint focuses on delivering a functional module that can be tested and improved.
The Agile workflow applied includes:

1. Requirement Analysis
2. System Design
3. Implementation
4. Testing
5. Deployment
6. Feedback and Iteration

This cycle repeats until the system reaches the desired level of performance and stability.

5.2 Phases of Agile Model

5.2.1 Requirement Gathering

This phase focuses on identifying and defining system requirements based on user needs and technical constraints.

Key Activities:

- Identifying supported input formats (PDF, text, YouTube URLs)
- Defining system outputs (MCQs, summaries, slides, knowledge maps)
- Selecting appropriate NLP models for each task
- Understanding user expectations (students, educators)
- Identifying performance and scalability constraints

Outcome:

A clearly defined set of functional and non-functional requirements.

5.2.2 System Design

This phase defines the architecture and structure of the system.

Key Activities:

- Designing frontend UI components (upload interface, dashboards, knowledge maps)
- Defining API communication between frontend and backend
- Designing database schema using MongoDB
- Structuring NLP processing pipelines
- Separating compute-heavy AI operations from standard API logic

Outcome:

A modular and scalable system architecture.

5.2.3 Implementation (Sprint Development)

In this phase, the system is developed incrementally through Agile sprints.

Key Activities:

- Developing frontend using React and Vite
- Implementing backend APIs using FastAPI
- Integrating NLP models (T5, DistilBART, FLAN-T5)
- Implementing document parsing and transcript extraction
- Connecting frontend, backend, and database

Sprint-Based Development Example:

- Sprint 1: File upload and parsing
- Sprint 2: Question generation
- Sprint 3: Summarization module
- Sprint 4: Socratic tutor
- Sprint 5: Knowledge map and UI refinement

Outcome:

Functional modules delivered incrementally.

5.2.4 Continuous Testing

Testing is performed throughout development to ensure system reliability and output quality.

Key Activities:

- Validating text extraction accuracy
- Evaluating NLP outputs (questions, summaries)
- Ensuring relevance and correctness of generated content
- Testing API responses and frontend integration
- Performing end-to-end system testing

Special Consideration:

Since AI outputs are not always deterministic, **manual evaluation and heuristic validation** are essential.

Outcome:

Improved accuracy, stability, and performance.

5.2.5 Deployment

Once a stable version is achieved, the system is deployed to production or testing environments.

Key Activities:

- Deploying frontend on platforms like Vercel or Netlify
- Hosting backend on cloud servers (AWS, GCP, DigitalOcean)
- Configuring APIs and CORS policies
- Monitoring system performance

Outcome:

Accessible and scalable system for users.

5.2.6 Maintenance and Iteration

After deployment, the system undergoes continuous improvement.

Key Activities:

- Fixing bugs and improving UI/UX
- Updating NLP models for better accuracy
- Optimizing performance and response time
- Adding new features based on user feedback

Outcome:

A continuously evolving and improving system.

5.3 Advantages of Agile for QueryQuill

The Agile model provides several advantages specifically suited to QueryQuill's architecture:

1. Continuous Model Improvement
AI models require frequent tuning. Agile enables iterative refinement of outputs.

2. Early Validation of Features
Core functionalities (e.g., MCQ generation) can be tested early before building advanced modules.

3. Parallel Development
Frontend and backend can be developed simultaneously, reducing development time.

4. Flexibility
New features (e.g., knowledge maps, video support) can be added without redesigning the system.

5. Risk Reduction
Technical risks (e.g., model performance, latency issues) are identified early and resolved incrementally.

6. Improved Quality
Continuous testing ensures higher reliability and better output quality.

5.4 Process Flow Summary

The Agile process in QueryQuill can be summarized as:

Requirement → Design → Develop → Test → Deploy → Improve → Repeat

This iterative loop ensures:

- Continuous delivery
- High adaptability
- Improved system quality over time

CHAPTER 6: PROJECT PLAN

This chapter outlines the structured plan for the development and deployment of QueryQuill. It defines project milestones, timelines, resource allocation, risk management strategies, quality assurance processes, and post-deployment support.

The project plan ensures that development progresses in a **controlled, measurable, and efficient manner**, aligned with Agile methodology.

6.1 Project Milestones

Milestones represent key checkpoints used to track progress and ensure timely completion of major phases.

Milestone	Description	Completion Date
Requirement Analysis	Finalization of system requirements and scope	10/08/2025
Design & Prototyping	UI/UX design and system architecture planning	25/08/2025
Development Phase	Frontend and backend implementation	10/11/2025
NLP Integration & Testing	Integration of AI models and validation	30/11/2025
Deployment & Final Review	System deployment and final evaluation	15/12/2025

Table 6.1 Project Milestones

6.2 Project Timeline

The project follows an Agile-based timeline, divided into phases with clearly defined objectives.

Phase	Duration	Key Activities
Requirement Analysis	25/07/2025 – 10/08/2025	Define inputs, outputs, and system scope
System Design	11/08/2025 – 25/08/2025	UI wireframes, API design, database schema
Development	26/08/2025 – 10/11/2025	Frontend, backend, and NLP integration
Testing & Integration	11/11/2025 – 30/11/2025	End-to-end validation and model tuning
Deployment & Maintenance	01/12/2025 onwards	Hosting, monitoring, and updates

Table 6.2 Project Timeline

6.3 Resource Allocation

Effective resource allocation ensures smooth execution across all stages.

6.3.1 Human Resources

Role	Responsibility
Frontend Developers	UI/UX development using React and Vite
Backend Engineers	API development using FastAPI
NLP/AI Engineers	Model integration and tuning

Table 6.3.1 Human Resources

6.3.2 Technical Resources

- Development Environment: Python (venv), Node.js, Uvicorn
- Tools: VS Code, Postman
- Version Control: Git, GitHub
- Client Interface: Web browsers (Chrome, Edge, Safari)

6.3.3 Financial Resources

- Cloud Hosting (AWS, DigitalOcean, etc.)
- Database Hosting (MongoDB Atlas)
- Optional APIs (OCR tools for scanned PDFs)

6.4 Risk Management

AI-based systems introduce unique technical and operational risks. These risks are identified and mitigated as follows:

6.4.1 Technical Risks

Risk	Description	Mitigation
NLP Processing Delay	Heavy models may slow response time	Use async processing and limit input size
Data Extraction Failure	Poor PDF or missing video transcripts	Provide user feedback and fallback options
AI Output Inaccuracy	Incorrect or irrelevant AI responses	Use prompt constraints and validation

Table 6.4.1 Technical Risks

6.4.2 Security Risks

Risk	Description	Mitigation
Data Privacy	Sensitive user uploads	Temporary processing, no permanent storage
API Abuse	Excessive requests	JWT authentication and rate limiting

Table 6.4.2 Security Risks

6.5 Quality Assurance (QA)

Quality assurance ensures system reliability, accuracy, and performance.

6.5.1 Testing Strategy

- **Unit Testing**
  Validate backend APIs and input processing modules

- **NLP Output Testing**
  Evaluate quality of generated questions and summaries
- **Integration Testing**
  Ensure smooth interaction between frontend, backend, and database

- **User Acceptance Testing (UAT)**
  Collect feedback from real users (students and educators)

6.5.2 Performance Metrics

Metric	Target
Inference Latency	< 15 seconds per 1000 words
Summary Accuracy	High human-evaluated relevance
UI Performance	Smooth interaction (~60 FPS)

Table 6.5.2 Performance Metrics

6.6 Maintenance and Support

Post-deployment, the system will be continuously monitored and improved.

6.6.1 System Maintenance

- Regular updates of NLP models
- Performance optimization
- Bug fixes and system patches
- Monitoring server load and scaling resources

6.6.2 User Support

- Help guides and tooltips
- Feature onboarding instructions
- Export and usage documentation
- Feedback collection for improvements

CHAPTER 7: SYSTEM DESIGN

System design is the process of translating requirements into an architectural blueprint. For QueryQuill, this blueprint governs how the complex Natural Language Processing (NLP) models, asynchronous backend logic, document-oriented database, and responsive React frontend interact harmoniously.

7.1 Design Strategy

QueryQuill employs a modern **3-Tier Architecture**, which cleanly separates responsibilities to ensure that massive NLP workloads do not block the user interface.

1. **Presentation Tier (Client-Side):** Developed using **React.js** and **Vite**, this tier manages all user interactions, visual mapping (Knowledge Maps), form submissions, and data display.
2. **Application Tier (Logic/Server-Side):** Developed using **Python** and **FastAPI**, this tier handles all the business logic. It securely processes file uploads, controls the asynchronous AI model pipelines (T5, DistilBART, FLAN-T5), and routes requests to the database.
3. **Data Tier (Database Server):** Developed utilizing **MongoDB**, this tier persistently stores non-relational JSON-like objects representing users, isolated questions, and compiled exams.

7.2 System Diagrams (UML)

The following Unified Modeling Language (UML) diagrams outline behavior and structural layouts of the QueryQuill platform.

7.2.1 USE CASE DIAGRAM

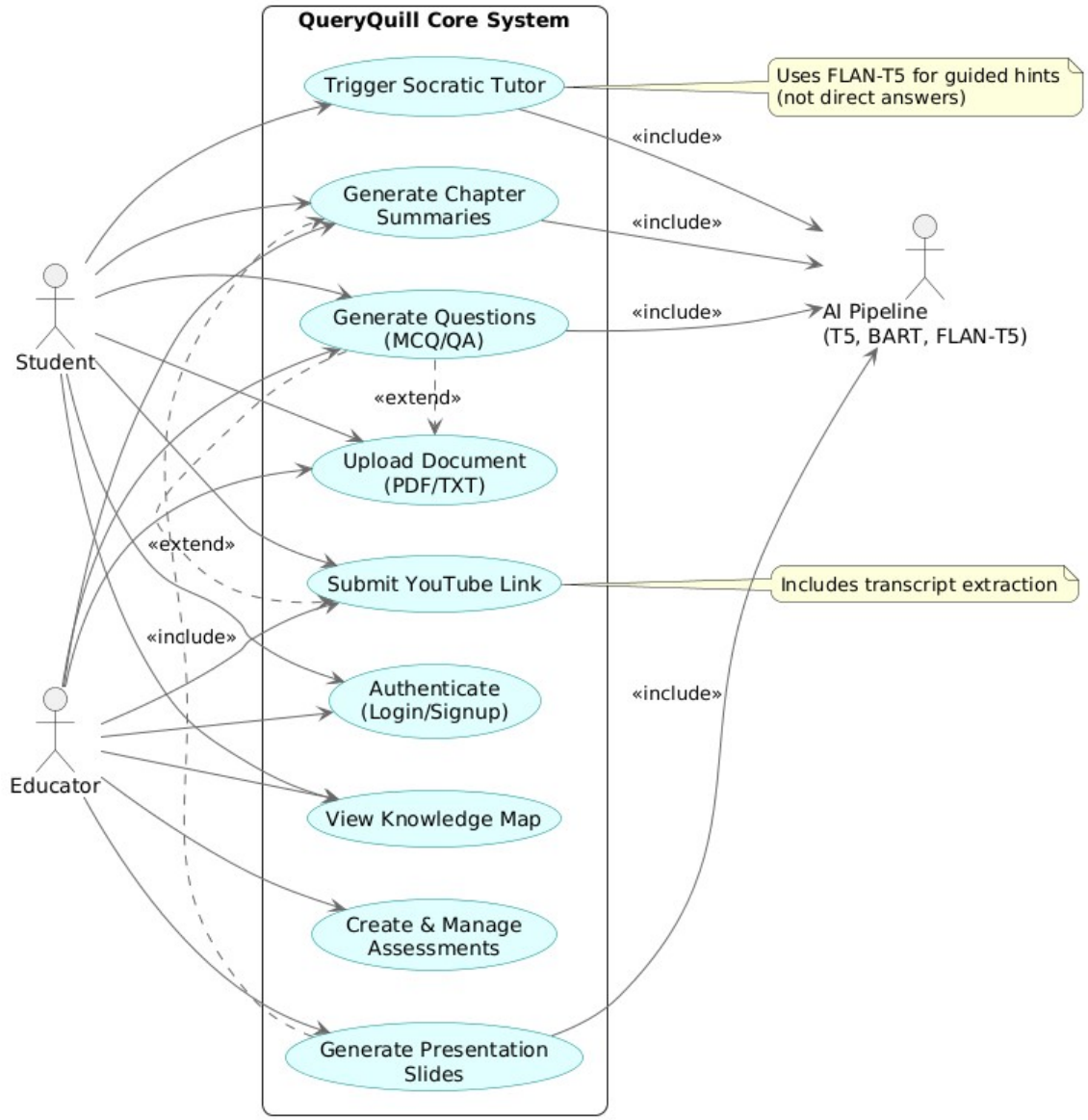


Figure 7.2.1 Use Case Diagram

7.2.2 CLASS DIAGRAM

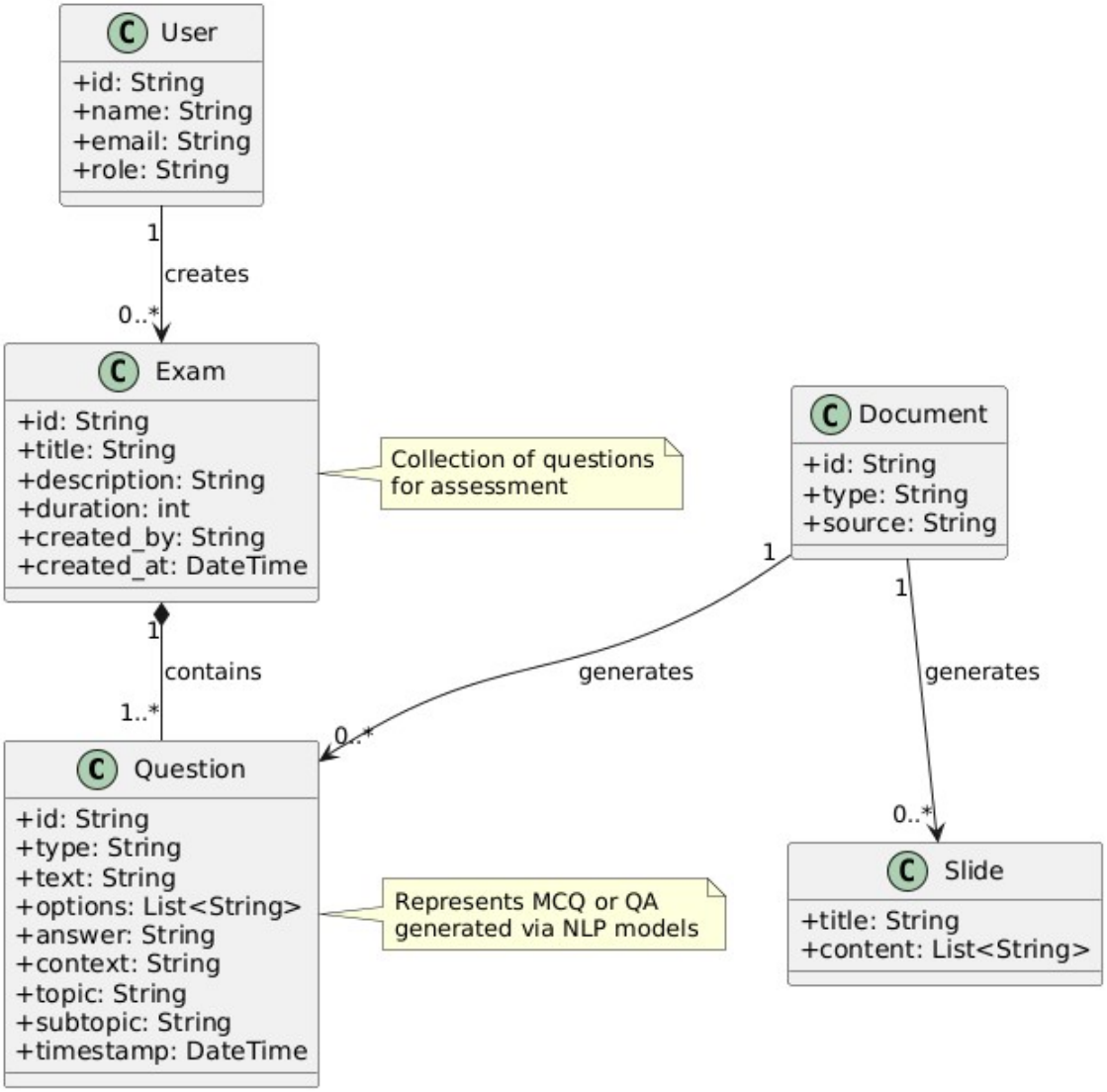


Figure 7.2.2 Class Diagram

7.2.3 SEQUENCE DIAGRAM

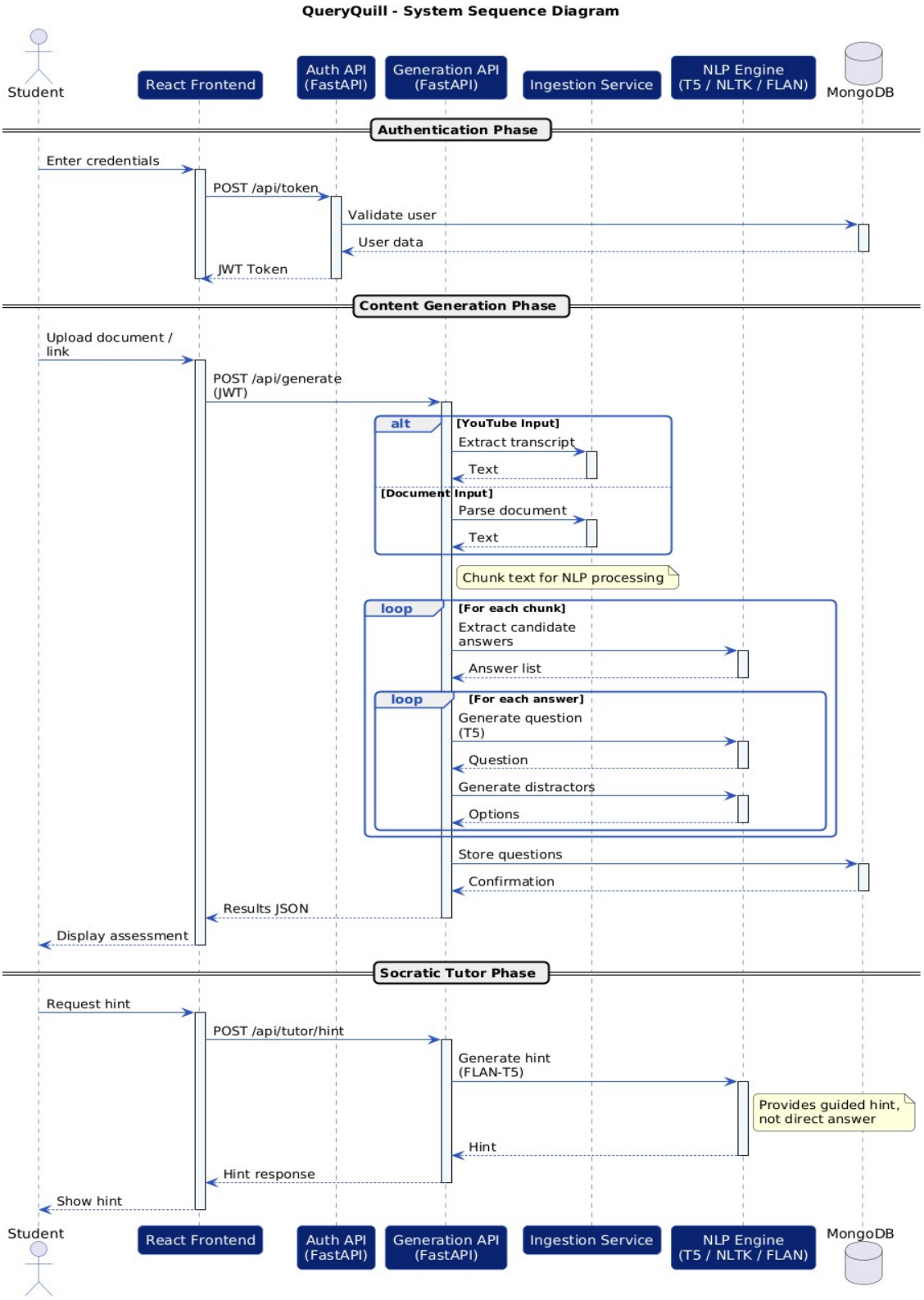


Figure 7.2.3 Sequence Diagram

7.2.4 ACTIVITY DIAGRAM

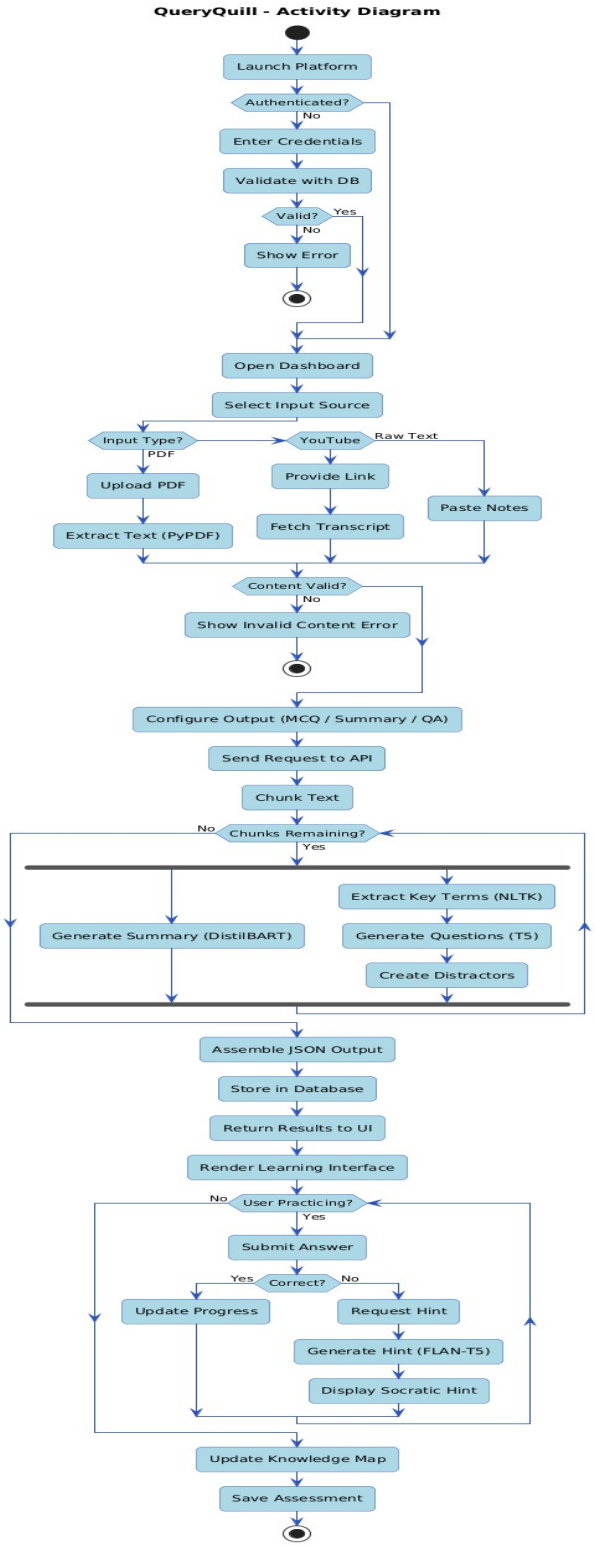


Figure 7.2.4 Activity Diagram

7.2.5 E-R DIAGRAM

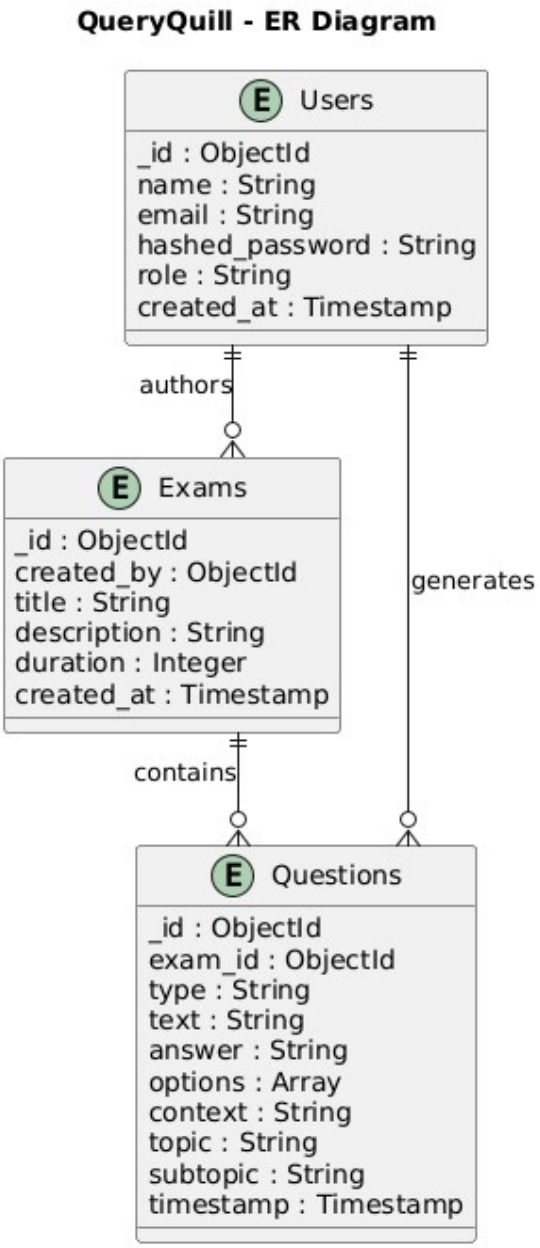


Figure 7.2.5 E-R Diagram

7.4 List of Tables & Data Dictionary

Because QueryQuill operates on a JSON-based NoSQL architecture (MongoDB), "Tables" are referred to as "Collections," and rows are "Documents."

7.4.1 Collection: users

Purpose: Secures and authenticates user access logic.

Field	Data Type	Constraint	Description
_id	ObjectId	Primary Key	Unique document identifier generated by Mongo.
name	String	Required	Full name of the user.
email	String	Unique, Required	Contact point and login credential.
hashed_password	String	Required	Bcrypt-hashed secure password string.
role	String	Default: student	Determines access controls (student/educator).
created_at	Datetime	Auto-Generated	Timestamp of registration.

Table 7.4.1 Collection: users

7.4.2 Collection: questions

Purpose: Stores globally generated quiz objects for the interactive Knowledge Map aggregation.

Field	Data Type	Constraint	Description
_id	ObjectId	Primary Key	Unique identifier.
type	String	mcq or qa	Determines frontend rendering layout.
text	String	Required	The generated question body.
options	Array[String]	Optional	Rendered distractors + correct answer (for MCQs).
answer	String	Required	The verified ground-truth answer.
context	String	Required	The original text chunk the question was derived from.
topic	String	Optional	Thematic category (e.g., "Computer Science").
subtopic	String	Optional	Granular category element (e.g., "Data Structures").

Table 7.4.2 Collection: questions

7.4.3 Collection: exams

Purpose: Saves grouped collections of generated questions into distinct evaluation modules

.

Field	Data Type	Constraint	Description
_id	ObjectId	Primary Key	Unique identifier.
title	String	Required	High-level name of the assessment.

description	String	Optional	Brief context about the exam focus.
questions	Array[Object]	Required	Nested array of Question schema documents.
duration	Integer	Default: 30	Time allocation for taking the exam (minutes).
created_by	String	Required	Foreign-key reference to the author's User ID/email.

Table 7.4.3 Collection: exams

7.5 Data Flow Diagram (DFD)

Data Flow Diagrams visually represent how context transitions through the system without outlining specific logic loops (like an Activity diagram does).

7.5.1 Level 0 DFD (Context Level)

The Context Diagram demonstrates the highest level of system boundary data flows.

- **Actor (User):** Sends raw data (PDF, YouTube URL, Authentication Credentials) into the **QueryQuill System**.
- **QueryQuill System:** Processes inputs and replies with requested outputs (Authentication Token, Generated JSON Questions, Knowledge Map visual Nodes).
- **External AI Boundary:** While mostly local, the system abstracts calls to Hugging Face models as its core processing block.

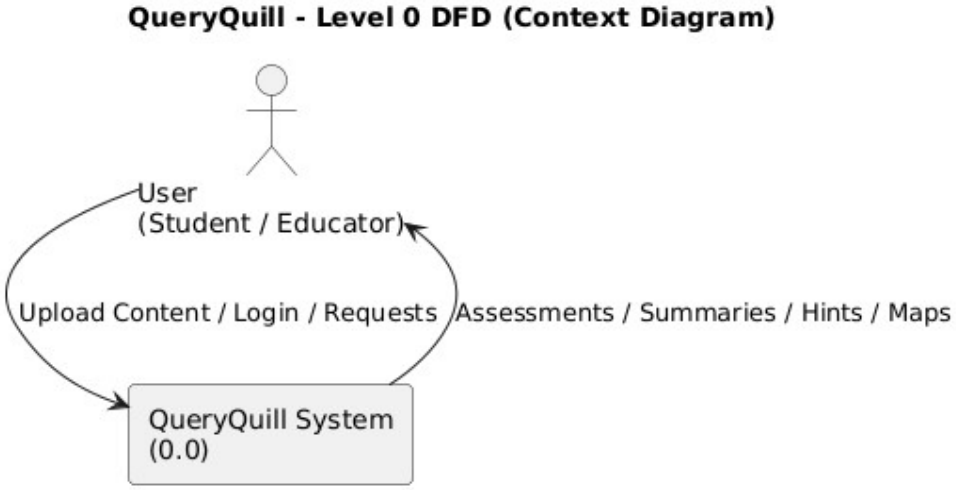


Figure 7.5.1 Level 0 DFD6

7.5.2 Level 1 DFD

The Level 1 DFD breaks down the Core System into discrete operational modules:

1. **Authentication Module:** Receives User Login Details → Validates with MongoDB User Collection → Returns JWT Bearer Token.

2. **Ingestion Preprocessor:** Receives JWT Token + PDF/YouTube Upload → Parses and sanitizes to raw text → Passes Text chunks to Generation Module.

3. **Core NLP Generation Module:** Receives raw text → Runs inference through T5/DistilBART/FLAN-T5 → Outputs logical JSON strings.

4. **Database Routing Module:** Intercepts logical JSON strings → Stores distinct attributes inside questions and exams collections.

5. **Presentation/Mapping Module:** Queries MongoDB for Topics/Subtopics → Calculates Node weights → Returns aggregated array links for React D3/Canvas map rendering to the User.

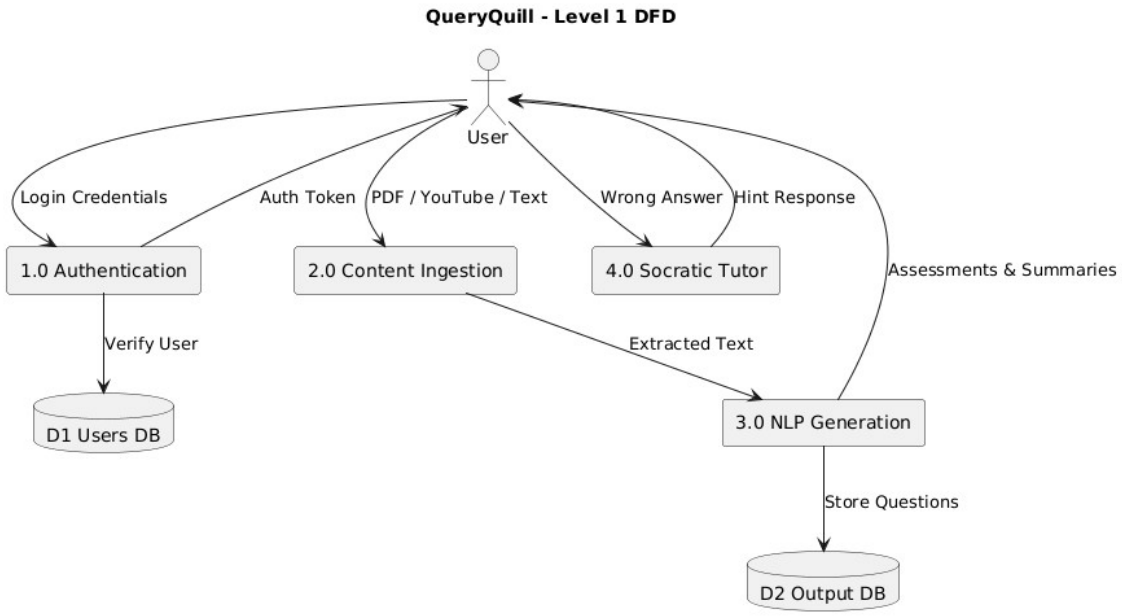


Figure 7.5.2 Level 1 DFD

7.5.3 Level 2 DFD

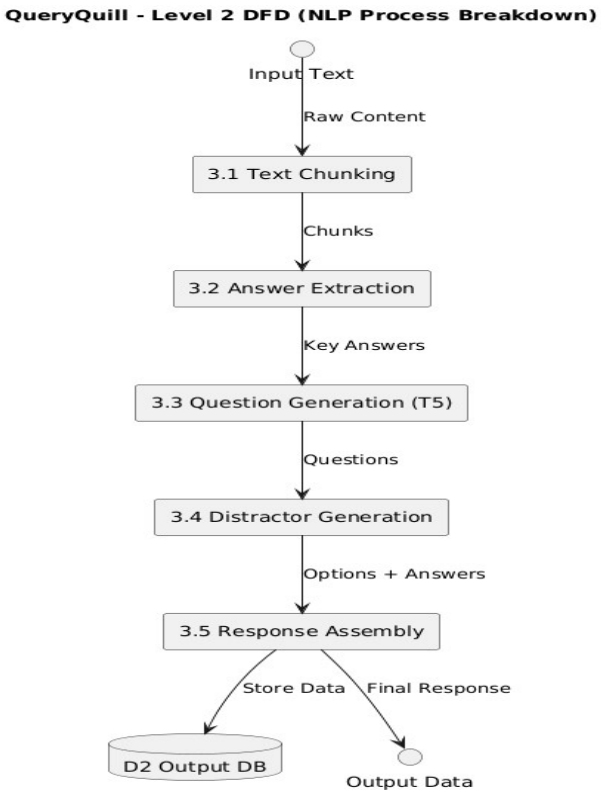


Figure 7.5.3 Level 2 DFD8

7.6 User Interface (UI) Design Outline

The User Interface strictly adheres to an aesthetic, distraction-free modern framework, promoting focus and reducing cognitive load. Standardized generic CSS frameworks like Tailwind provide a glass-morphic or clean minimalistic view.

1. **Login/Registration Form Layout:**
    - Centered semantic card overlay on a branded aesthetic background.
    - Fields: Name, Email (validated), Password (masked), and Role Dropdown.
    - Error mapping highlights fields red dynamically if MongoDB detects duplicate emails.

2. **Student / Educator Main Dashboard:**
    - **Side Navigation Bar:** Links to "New Upload", "My Exams", "Knowledge Map", and "Profile".
    - **Main Window View:** A Drag-And-Drop interface for generic text/PDF files, alongside a dedicated input-bar for YouTube URLs.
    - *Report Layout:* The center panel lists previous generated items as selectable cards showing Title, Question Count, and Date.

3. **Assessment Interactive Layout:**
    - Displays one question at a time to prevent cognitive overload.
    - Features a persistent lateral modal triggering the **Socratic Tutor**. When clicked, it passes the context state to the FLAN-T5 endpoint.

4. **Knowledge Map Visual Rendering:**
    - A full-screen, interactive canvas (e.g., relying on react-force-graph).
    - Nodes represent distinct topics (larger nodes = more questions generated under that topic). Linked lines map conceptual relationships across subtopics intuitively.

CHAPTER 8: IMPLEMENTATION DETAILS

This chapter details the exact programmatic execution of the QueryQuill system. It defines the core procedural logic governing the Artificial Intelligence and displays the specific code snippets responsible for the system's most complex features.

8.1 Algorithm of Implementation

The primary generation methodology follows a sequenced, context-aware Natural Language Processing pipeline:

1. **Initialize Architecture:** Boot the FastAPI ASGI server (Uvicorn) and allocate the Hugging Face Pipeline models (valhalla/t5-base-qg-hl and sshleifer/distilbart-cnn-12-6) into the available RAM.

2. **Accept HTTP Request:** The server receives a POST /api/generate request containing an authorization JWT token, alongside either a raw PDF byte-stream or a targeted YouTube URL string.

3. **Data Extraction:**
   - *If PDF:* Instantiate PyPDF.PdfReader to extract pages into a unified /n delimited string.
   - *If YouTube URL:* Parse the Video ID and instantiate YouTubeTranscriptApi to download the synchronized caption string.

4. **Semantic Text Chunking:** Validate string length and slice the full document into localized, semantic chunks (~400/500 tokens) to respect the attention window limits of the T5 transformer model.

5. **Identify Ground Truth Answers:** For each processed text chunk, instantiate the NLTK pos_tag (Part-Of-Speech) algorithm to highlight proper noun phrases (NN, NNS, NNP). These target strings are saved as candidate_answers.

6. **Execute Question Synthesis:** For each candidate_answer, concatenate it with its parent chunk as context ("answer: {ans} context: {ctx}") and feed the normalized payload into the T5 beam-search generator, instructing the model to output a grammatically correct interrogative sentence.

7. **Generate Invalid Distractors:** For each successful question generation, call the NLTK engine again on the same parent chunk to select 3 semantically related but factually alternative nouns to serve as wrong_options.

8. **Deduplicate Output:** Filter the newly generated question against previous generations using Python's difflib.SequenceMatcher. If linguistic similarity ratio > 0.85, discard to prevent redundant quiz objects.

9. **Format Payload:** Compile the validated question string, the target ground-truth answer, and the shuffled distractors into a Python Dictionary mapping.

10. **Database Insertion:** Serialize the list of dictionary mappings into BSON and execute an asynchronous insert-many command to the MongoDB questions collection (db.questions.insert_one()).

11. **Return Visual Output:** Return an HTTP 200 JSON Response payload. The React/Vite frontend maps over the returned array to render distinct UI question cards and aggregate Knowledge Map linking nodes.

8.2 Flowchart of Implementation

The following procedural flowchart represents the physical constraints and logic branches of the Question Generation Algorithm. *(Copy this code block into PlantUML to render the visual flowchart).*

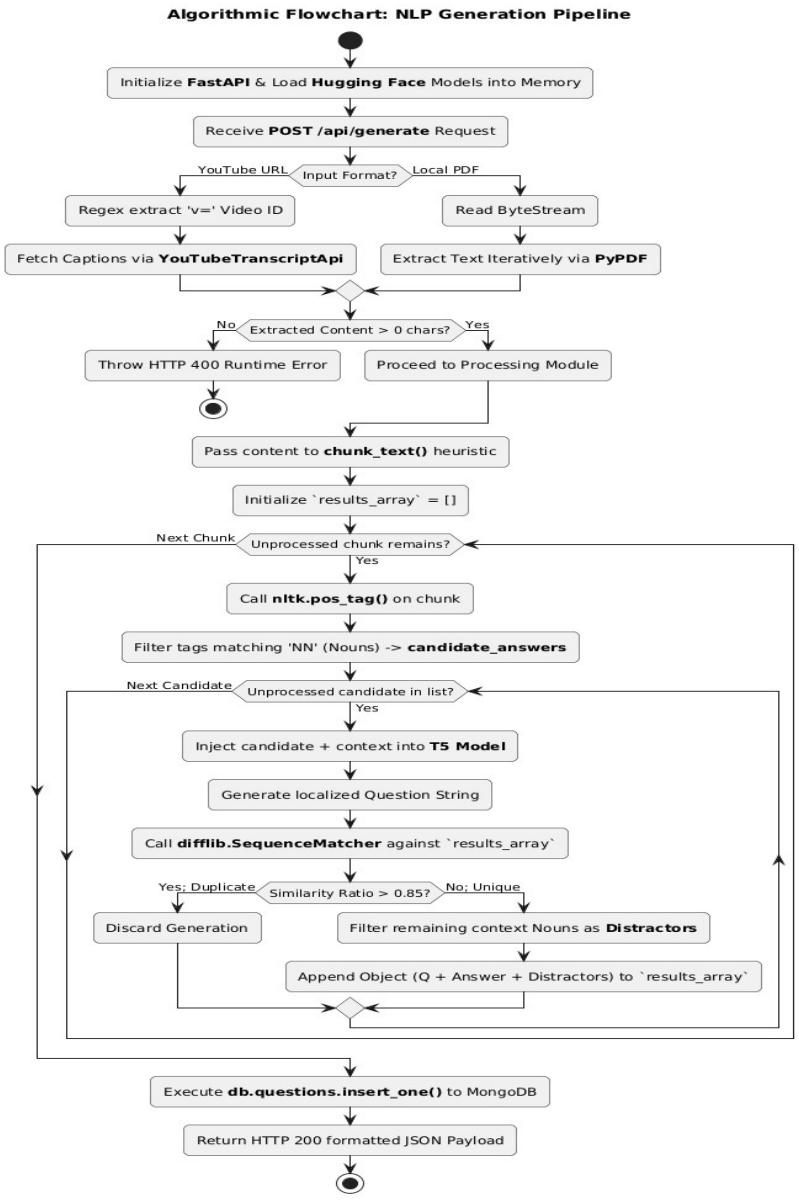


Figure 8.2 Flowchart of Implementation

8.3 Actual Program Code

The following snippets demonstrate exactly how the abstracted algorithms are syntactically implemented within the Python FastAPI backend.

8.3.1 NLTK Heuristic Answer Identification

Located inside src/mcq_engine.py This snippet represents how the MCQEngine identifies the ground-truth answers prior to running the Heavy Transformer. By filtering specific Part-of-Speech (POS) tags (Nouns), the system determines what the focus of the AI question should be.

```
import nltk
from collections import Counter
class MCQEngine:
    def get_candidate_answers(self, text: str, num_candidates: int = 5) -> list[str]:
        """
        Extracts nouns/noun phrases from text to serve as answers.
        Returns a list of unique strings.
        """
        sentences = nltk.sent_tokenize(text)
        candidates = []

        for sent in sentences:
            words = nltk.word_tokenize(sent)
            tags = nltk.pos_tag(words)

            # Simple heuristic: filter for Nouns (NN, NNS, NNP, NNPS)
            # Avoid single character words or very common stop words
            nouns = [word for word, pos in tags if pos.startswith('NN') and len(word) > 2]
candidates.extend(nouns)

        # Return a randomly sampled unique list to ensure output variety
unique_candidates = list(set(candidates))

        if len(unique_candidates) >num_candidates:
            return random.sample(unique_candidates, num_candidates)
        return unique_candidates
```

8.3.2 Hugging Face T5 NLP Inference Loop

Located inside src/generator.py This code details the interaction with the underlying architecture of the Hugging Face transformer pipeline. It encodes the specialized prompt into PyTorch Tensors and decodes the beam-search predictions.

```
import torch
from transformers import AutoTokenizer, AutoModelForSeq2SeqLM
class QuestionGenerator:
    def __init__(self, model_name="valhalla/t5-base-qg-hl"):
        """Initializes the T5 model for Question Generation."""
self.tokenizer = AutoTokenizer.from_pretrained(model_name)
self.model = AutoModelForSeq2SeqLM.from_pretrained(model_name)
```

```
self.device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
        self.model.to(self.device)
    def generate_for_answer(self, answer: str, text_context: str) -> str:
        """Generates a single question heavily anchored to the passed correct 'answer'
string."""
input_text = f"answer: {answer} context: {text_context}"

        # Hardware-optimized tensor encoding
        inputs = self.tokenizer.encode(
input_text,
return_tensors="pt",
max_length=512,
            truncation=True
        ).to(self.device)
        # Beam-Search configuration
        outputs = self.model.generate(
            inputs,
max_length=64,
num_beams=5,
do_sample=True,
top_k=50,
top_p=0.95,
num_return_sequences=1
        )
        return self.tokenizer.decode(outputs[0], skip_special_tokens=True)
```

8.3.3 Socratic Tutor Endpoint (FLAN-T5)

Located inside server.py and src/tutor.py This snippet highlights the implementation of an interactive REST route evaluating incorrect student input and executing the FLAN-T5 model to print a contextual learning hint dynamically instead of just revealing the rigid answer.

```
@app.post("/api/tutor/hint")
async def get_tutor_hint(request: HintRequest):
    # Ensures the model loaded completely over Uvicorn startup before responding
    if not tutor_model:
        raise HTTPException(status_code=503, detail="Tutor model is not loaded yet.")

    # Executes the Socratic prompting class logic using pipeline text2text-generation
    hint = tutor_model.generate_hint(
request.question_text,
request.wrong_answer,
request.correct_answer,
request.context
    )

    # Return JSON to React UI
    return {"hint": hint}
```

CHAPTER 9: TESTING

Testing ensures that the system functions correctly and delivers expected outputs. Due to the AI-driven nature of QueryQuill, testing focused primarily on validating overall system behavior rather than strict deterministic outputs.

9.1 Testing Approach

The following basic testing methods were used:

- **Black-Box-Testing**
  Verified system functionality through user inputs without focusing on internal implementation.
- **Integration-Testing**
  Ensured proper communication between frontend (React) and backend (FastAPI), especially for API responses and data flow.
- **Manual-Validation(AI-Outputs)**
  Generated questions and summaries were manually checked for basic correctness and relevance.

9.2 Basic Test Cases

Test Case ID	Feature	Description	Result
TC-01	Authentication	Login with valid credentials	Pass
TC-02	Authentication	Login with invalid credentials	Pass
TC-03	Ingestion	Upload PDF / YouTube link	Pass
TC-04	Generation	Generate questions from content	Pass
TC-05	Socratic Tutor	Hint generation on wrong answer	Pass

9.3 Result Summary

The system performed as expected for basic functionalities including authentication, content processing, and output generation. Minor variations in AI-generated outputs are expected due to the nature of NLP models.

CHAPTER 10: USER MANUAL

10.1 System Installation & Setup

QueryQuill follows a client-server architecture with separate setup for backend and frontend.

Prerequisites

- Python 3.9+
- Node.js 18+
- MongoDB (Local or Atlas)

Step 1: Backend Setup

```
cd backend
python -m venvvenv
venv\Scripts\activate   # Windows
pip install -r requirements.txt
```

Create .env file:

```
MONGODB_URI=your_mongodb_uri
JWT_SECRET_KEY=your_secret_key
ALGORITHM=HS256
ACCESS_TOKEN_EXPIRE_MINUTES=30
```

Run backend server:

```
uvicornserver:app --reload
```

Step 2: Frontend Setup

```
cd frontend
npm install
npm run dev
```

Open in browser:

http://localhost:5173

10.2 System Usage

1. **Login / Signup**
   - Enter email and password
   - System verifies credentials
   - JWT token is generated for session

2. **Upload Content**
   Users can:
   - Upload PDF/TXT files
   - Enter YouTube video link

   System extracts and processes text automatically.
3. **Generate Output**

User selects required output:
- MCQs / Questions
- Summaries
- Slides

System processes data using NLP models and returns results.

4. **Attempt Assessment**
   - Questions are displayed interactively
   - User selects answers
   - System tracks performance

5. **Socratic Tutor**
   - Triggered on incorrect answers
   - Provides hints (not direct answers)
   - Helps improve understanding

6. **Knowledge Map**
   - Displays topics and subtopics visually
   - Helps identify learning gaps

10.3 Application Screenshots

The following screenshots illustrate the key screens and workflows of the QueryQuill application, covering both the student-facing and teacher-facing interfaces.

10.3.1 Login / Sign Up Page

The login page is the entry point for all users. It presents a clean, minimal interface where users can sign in using their registered email and password. Test credentials are displayed for demonstration purposes. New users can navigate to account creation via the "Create Account" link. Upon successful authentication, the system generates a JWT token and redirects the user to their respective dashboard based on their role (student or teacher).

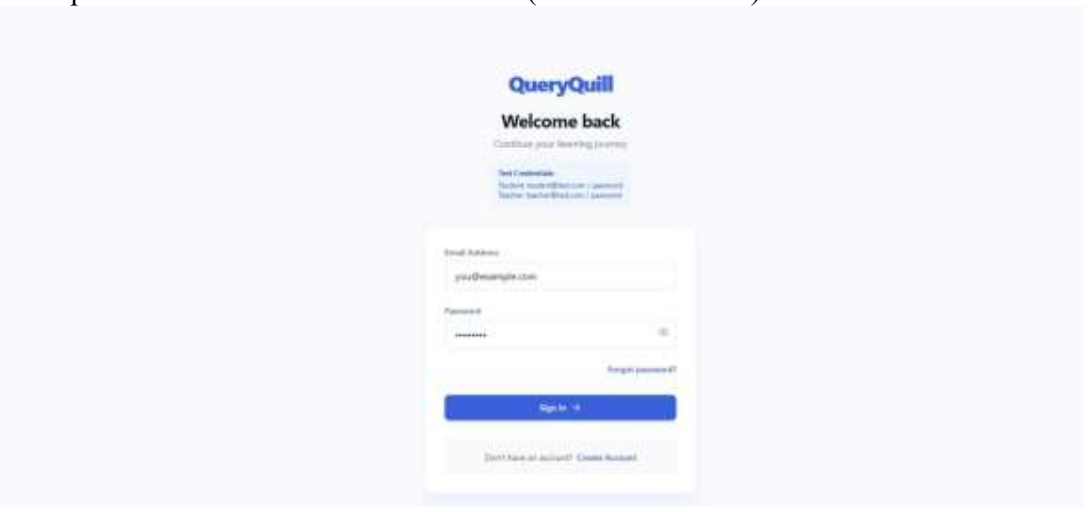


Figure 10.3.1 Login/SignUp Page

10.3.2 Student Dashboard

The student dashboard serves as the central hub for learners. It displays key progress metrics including Daily Streak, Total XP, Quizzes Taken, and Study Hours. The "Continue Learning" section shows the most recent study material (e.g., React Fundamentals) with a chapter progress bar, alongside any pending quizzes with their due dates. An Achievements panel encourages engagement through gamification badges. Quick-action buttons (View My Brain, Quick Summary, Start Practice) are accessible from the top bar.

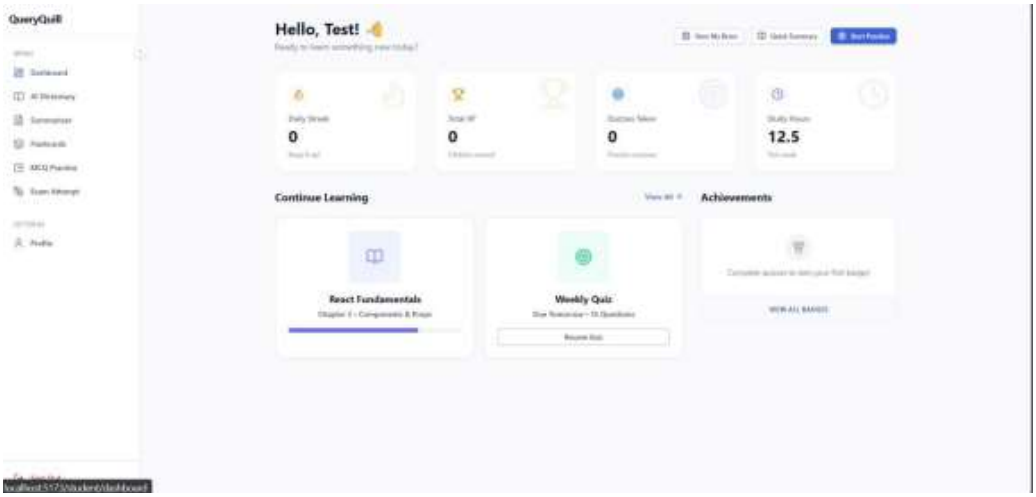


Figure 10.3.2 Student Dashboard

10.3.3 Teacher Dashboard

The teacher dashboard provides instructors with a comprehensive overview of their classroom. Summary cards display Total Students (48, up 12% from last month), Active Exams, Average Score (85% — Top 5% of region), and Slides Created (14). The "Exclusive Teacher Tools" section offers quick access to the AI Exam Generator (for auto-generating MCQs and short/long answer questions from PDF notes) and the PDF to Slides converter. A "Recent Exam Papers" section with search functionality is displayed below.

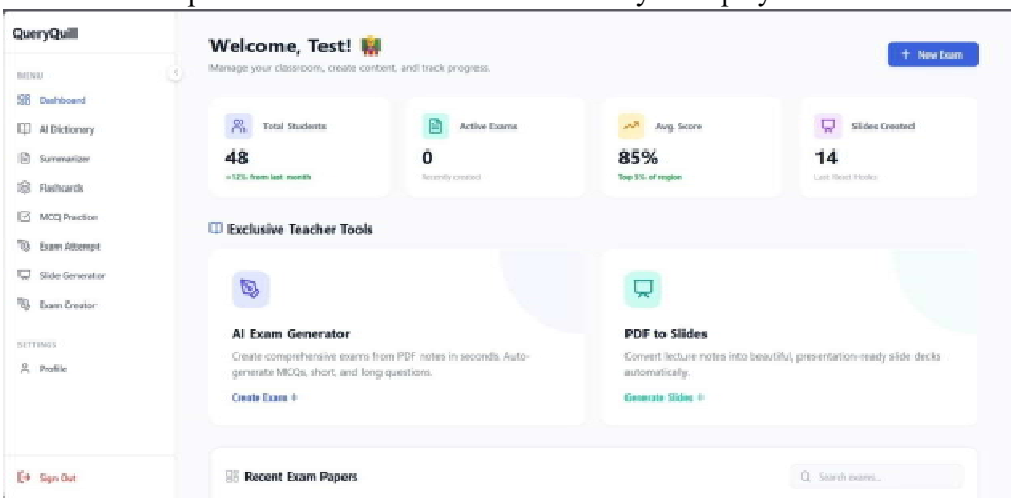


Figure 10.3.3 Teacher Dashboard

10.3.4 AI Dictionary

The AI Dictionary feature allows users to look up any word and receive an AI-generated definition, phonetic pronunciation, word families, and an Etymology & History panel. In the example shown, the word "education" is defined as a noun meaning "the process of imparting knowledge, skill and judgment," with related word families (educate, didactics, breeding, etc.) shown as tag chips. The right panel tracks recent lookups for quick reference. This tool supports vocabulary enrichment directly within the learning platform.

Figure 13.4 AI Dictionary

10.3.5 AI Summarizer

The AI Summarizer transforms long documents and video lectures into concise key insights. Users can upload a PDF, paste plain text, or provide a YouTube lecture URL. Once a file is uploaded (shown here: test_book.txt, 1.4 KB), clicking "Generate Insight" triggers the NLP pipeline. The resulting Key Insights panel on the right displays a structured, readable summary of the material — in this case, summarising the Data Link Layer (DLL) from a computer networking document. This feature significantly reduces study time for dense academic content.

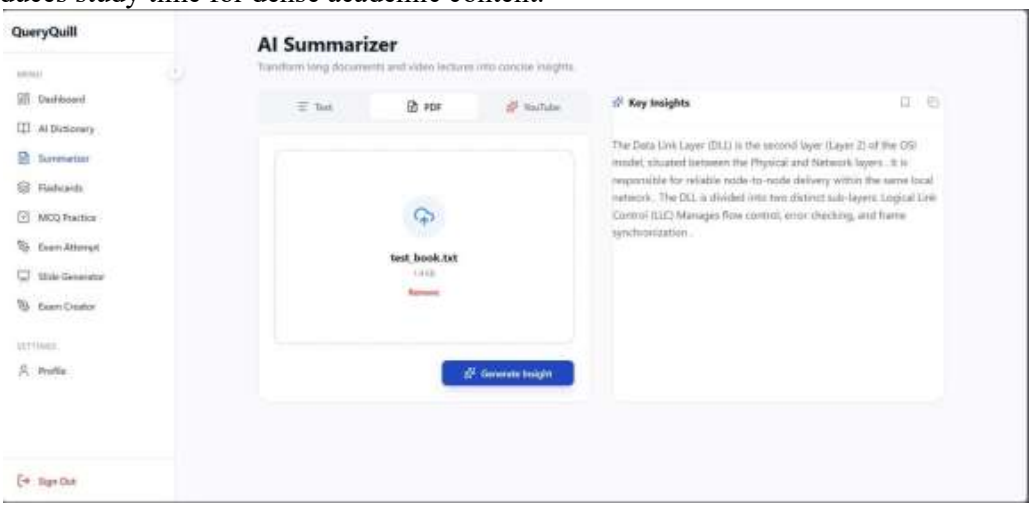
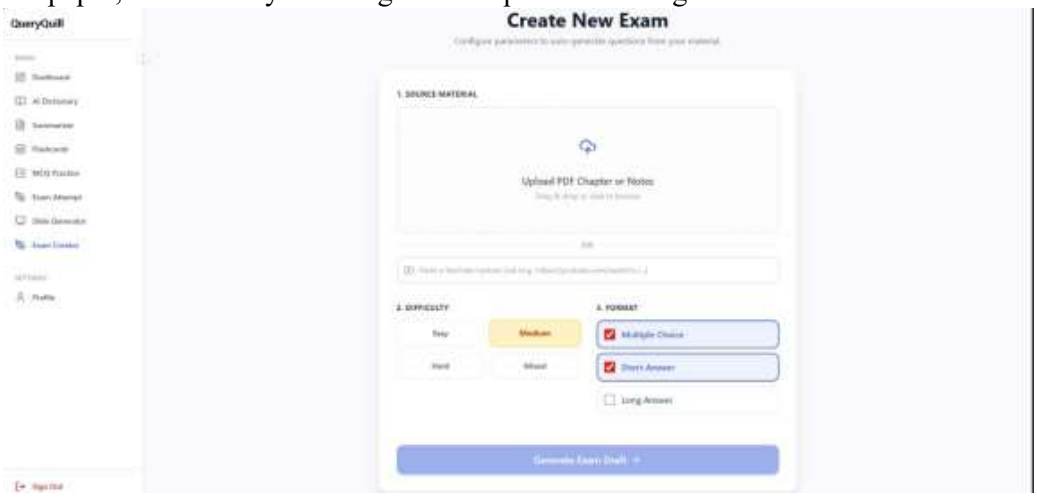


Figure 10.3.5 AI Summarizer

10.3.6 Exam Creator

The Exam Creator is an exclusive teacher tool that auto-generates exam questions from uploaded study material. Instructors can upload a PDF chapter or notes (via drag-and-drop or file browse), or paste a YouTube lecture URL as the source material. They can then configure the difficulty level (Easy, Medium, Hard, or Mixed) and select question formats (Multiple Choice, Short Answer, and/or Long Answer). Clicking "Generate Exam Draft" invokes theAI pipeline to produce a ready-to-use exam paper, dramatically reducing manual question-setting effort.



10.3.7 Exam Attempt Portal

The Exam Attempt Portal allows students to take assigned exams by entering their unique Exam Session ID provided by their instructor. The interface is intentionally simple and distraction-free: a single input field prompts for the session code (e.g., EX-2025-001), and a prominent "Start Exam" button initiates the assessment. This controlled entry ensures exam integrity, as only students with a valid session ID can access the corresponding exam paper.

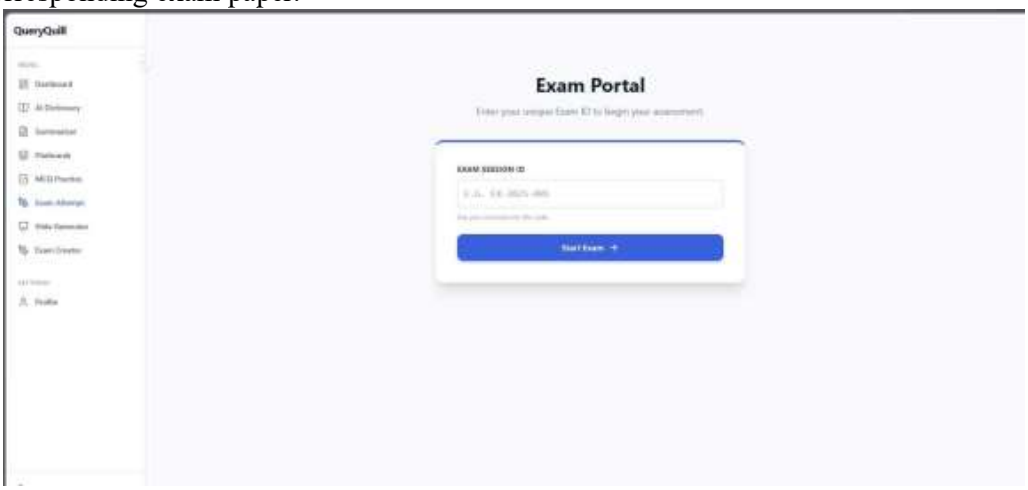


Figure 10.3.7 Exam Attempt Portal

CHAPTER 11: CONCLUSION AND FUTURE WORK

11.1 Conclusion

QueryQuill demonstrates the practical application of Artificial Intelligence in transforming traditional learning into an interactive and efficient process. In conventional educational systems, learners rely heavily on static resources such as textbooks, PDFs, and video lectures, which often require significant manual effort to extract meaningful insights and create practice material.

The system successfully addresses this limitation by converting passive content into structured, interactive outputs such as assessments, summaries, and knowledge maps. By integrating multiple specialized NLP models—including T5 for question generation, DistilBART for summarization, and FLAN-T5 for guided tutoring—QueryQuill ensures that learning becomes active, personalized, and concept-driven.

The implementation of a decoupled architecture using React/Vite for the frontend and FastAPI for the backend further validates the scalability and performance of the system. Additionally, the Socratic Tutor feature highlights a key advancement by promoting critical thinking through hints rather than direct answers, aligning with modern educational methodologies.

Overall, QueryQuill proves that complex AI-driven processes can be effectively abstracted into a user-friendly platform, enhancing both learning efficiency and teaching productivity. The system not only simplifies content consumption but also improves retention, engagement, and conceptual understanding.

11.2 Future Enhancements

Although QueryQuill achieves its core objectives, there are several opportunities for further improvement and scalability:

- **Advanced Language Models Integration**
  Upgrading to more powerful models (e.g., LLaMA, Mistral) can improve contextual understanding and enable generation of deeper, analytical questions.

- **OCR Integration**
  Incorporating Optical Character Recognition will allow the system to process scanned documents and handwritten notes, expanding input flexibility.

- **Gamification and Collaborative Learning**
  Introducing features such as leaderboards, study groups, and real-time quiz competitions can increase engagement and motivation.

- **Automated Evaluation of Descriptive Answers**
  Extending the system to evaluate long-form answers using semantic similarity techniques can provide detailed feedback beyond objective questions.

- **Spaced Repetition System (SRS)**
  Implementing adaptive revision techniques will help users revisit weak topics at optimized intervals, improving long-term retention.

CHAPTER 12: ANNEXURE

12.1 Glossary of Terms and Abbreviations

- **API (Application Programming Interface):** Enables communication between frontend and backend systems.
- **DFD (Data Flow Diagram):** Represents how data moves through the system.
- **ERD (Entity-Relationship Diagram):** Defines relationships between data entities such as Users, Exams, and Questions.
- **JSON (JavaScript Object Notation):** Format used for data exchange between system components.
- **JWT (JSON Web Token):** Secure token used for authentication and authorization.
- **MCQ (Multiple Choice Question):** Question format with multiple options and one correct answer.
- **NLP (Natural Language Processing):** AI domain used to process and generate human language.
- **POS Tagging:** Technique used to identify grammatical roles (noun, verb, etc.) in text.
- **QA (Question Answering):** Format where users provide descriptive answers.
- **SRS (Software Requirements Specification):** Document defining system requirements and design.
- **OOM (Out Of Memory):** Error when system exceeds available memory during processing.
- **HITL (Human-in-the-Loop):** Approach where human validation improves AI outputs.

12.2 References

The development of QueryQuill is supported by the following official resources and documentation:

[1] FastAPI Documentation: https://fastapi.tiangolo.com/
[2] React Documentation: https://react.dev/
[3] Hugging Face Transformers: https://huggingface.co/docs/transformers/
[4] NLTK Documentation: https://www.nltk.org/
[5] PyMongo Documentation: https://pymongo.readthedocs.io/
[6] IEEE SRS Standard (IEEE 830-1998)

12.3 Tools and Technologies

The system is built using a modern, scalable technology stack:

- **Python (3.9+):** Core backend language for AI and logic processing
- **FastAPI:** High-performance backend framework with asynchronous support
- **React + Vite:** Frontend framework for building dynamic user interfaces
- **Tailwind CSS:** Utility-based styling for responsive UI design
- **PyTorch:** Framework for executing machine learning models
- **Hugging Face Transformers:** Used for T5, DistilBART, and FLAN-T5 models
- **NLTK:** Used for text processing and distractor generation
- **MongoDB:** NoSQL database for storing flexible JSON-like data

12.4 About the College

U.V. Patel College of Engineering (UVPCE) is one of the premier engineering institutes in North Gujarat, established under Ganpat University. The college offers undergraduate and postgraduate programs across multiple engineering disciplines including Computer Engineering, Information Technology, Electronics, Mechanical, and Civil Engineering.

The QueryQuill – A Smart Study Buddy was developed as a CP-II Project under the Computer Engineering Department of U.V. Patel College of Engineering, Ganpat University.

For complete and up-to-date information about the college - including departments, faculty, courses, facilities, and achievements - please visit the official college website:

Official Website: https://uvpce.guni.ac.in/